Lecture Notes 02: Intuitive Matrix Algebra Through Traffic Networks

AI 501: Mathematics for Artificial Intelligence

Fall 2026

Abstract

This guide builds your intuition for matrix operations through practical, real-world scenarios. Using Lahore's traffic monitoring network alongside financial and geometric examples, we explore how matrices store multi-variable data, transform vectors, combine signals, and undo transformations.

Contents

1 Building Matrices from Real-World Data **2**

2 Visualizing Matrix Operations in Action **2**
2.1 Combining and Scaling Networks 2
2.2 Changing Perspectives: The Transpose 3
2.3 Measuring Whole-System Magnitude 3
2.4 The Trace: A Diagonal Summary 3
2.5 Symmetric and Skew-Symmetric Matrices 4

3 Understanding Matrix-Vector Multiplication **5**
3.1 Perspective 1: Row-wise Scoring and Aggregation 5
3.2 Perspective 2: Column-wise Pattern Blending 5
3.3 Perspective 3: Feature Matrices and Weight Vectors in Machine Learning 6

4 Matrix Rank, Column Space, and System Reachability **6**
4.1 The Column Space: Everything Matrix A Can Reach 6
4.2 Matrix Rank and Column Redundancy 7
4.3 Matrix Basis and Dimension of Column Space 7
4.4 Connection to Linear Systems and Matrix Inversion 7

5 Mastering Matrix Multiplication **8**
5.1 The Dimension Matching Rule 8
5.2 Comparing Feature Profiles: The Gram Matrix 9

6 Matrices as Linear Transformations **9**
6.1 Intuitive Systems Thinking 9
6.2 Common Transformation Behaviors 10

7 Inverting Systems: Undoing Transformations **10**
7.1 The Matrix Inverse Concept 10
7.2 Worked Inverse Scenario 10
7.3 When Is Inversion Impossible? Understanding Singular Matrices 10
7.4 The Determinant: Measuring Area and Volume Scaling 11

8 Matrix Inverses: Left, Right, and Pseudo-Inverses **11**
8.1 Left-Inverse (Tall Matrices: More Equations than Unknowns) 11
8.2 Right-Inverse (Wide Matrices: More Unknowns than Equations) 12
8.3 Standard Two-Sided Inverse (Square & Nonsingular) 12
8.4 Special Structured Invertible Matrices 12
8.5 Orthogonal and Orthonormal Matrices 13
8.6 The Pseudo-Inverse: A Universal "Best Undo," $M^\dagger$ 13

9 Summary Map **14**

1 Building Matrices from Real-World Data

In our previous discussion, we used a single 4-dimensional vector to describe queue lengths at a single intersection (North, East, South, West). But imagine standing inside Lahore's central traffic management hub. The city doesn't just manage one junction; it manages hundreds simultaneously.

Suppose the control center actively monitors 3 key intersections. Stacking their individual 4-approach queue vectors together gives us a grid of numbers $A \in \mathbb{R}^{3\times 4}$:

$$A = \begin{pmatrix} 30 & 18 & 25 & 12 \\ 22 & 10 & 15 & 8 \\ 40 & 25 & 30 & 20 \end{pmatrix}.$$

Notice what happens when we arrange data this way:

- **Looking across a row:** Row 1 gives us the complete picture of Intersection 1 across all four approaches $(30, 18, 25, 12)$.
- **Looking down a column:** Column 3 isolates Southbound traffic across all three intersections $(25, 15, 30)^\mathsf{T}$.

A single index is no longer enough to pinpoint a piece of data. To locate 15, we look at **Row 2, Column 3** (A_{23}). Matrices are simply organized 2D grids ($m \times n$) that let us process multi-entity, multi-variable systems all at once.

2 Visualizing Matrix Operations in Action

Instead of memorizing mechanical rules, let's see how matrix operations represent physical changes in our traffic network.

2.1 Combining and Scaling Networks

Suppose matrix A represents queue counts at 8:00 AM. Over the next 20 minutes, sensors log the net change in vehicles across all approaches, captured in matrix $B \in \mathbb{R}^{3\times 4}$:

$$B = \begin{pmatrix} 4 & -3 & -5 & 2 \\ 1 & 2 & -1 & 0 \\ -2 & 4 & 3 & -1 \end{pmatrix}.$$

Adding $A + B$ updates the entire city grid simultaneously:

$$A + B = \begin{pmatrix} 34 & 15 & 20 & 14 \\ 23 & 12 & 14 & 8 \\ 38 & 29 & 33 & 19 \end{pmatrix}.$$

What if traffic increases uniformly by 10% across the city due to rain? We simply scale the matrix by $1.1A$, multiplying every single entry by 1.1.

2.2 Changing Perspectives: The Transpose

Transposing a matrix (A^T) flips it along its main diagonal, turning rows into columns and columns into rows. Our original 3×4 matrix becomes a 4×3 matrix:

$$A^\mathsf{T} = \begin{pmatrix} 30 & 22 & 40 \\ 18 & 10 & 25 \\ 25 & 15 & 30 \\ 12 & 8 & 20 \end{pmatrix}.$$

The underlying information hasn't changed—we are simply shifting our view from *"intersections with approach features"* to *"approaches compared across intersections."*

2.3 Measuring Whole-System Magnitude

How do we calculate a single number that reflects the overall congestion level across the entire network?

The **Frobenius norm** ($\|A\|_F$) extends the standard Euclidean distance to grids by squaring every entry, adding them all up, and taking the square root. For our change matrix B:

$$\|B\|_F = \sqrt{4^2 + (-3)^2 + (-5)^2 + \cdots + (-1)^2} = \sqrt{74} \approx 8.6.$$

This single value acts as a network-wide metric for total traffic fluctuation.

Check yourself. If every queue in matrix A doubles, what happens to the overall network fluctuation norm $\|2A\|_F$?

Answer. It doubles ($2\|A\|_F$). Scaling all entries uniformly scales the aggregate norm by the exact same factor.

2.4 The Trace: A Diagonal Summary

Some matrices are square, and for these we can compute one more useful summary: the **trace**, written $\mathrm{tr}(M)$, which is simply the sum of the entries sitting on the main diagonal — everything else in the matrix is ignored.

For example, if a small 2×2 matrix records how a signal-timing adjustment mixes with itself,

$$M = \begin{pmatrix} 5 & 2 \\ 1 & 3 \end{pmatrix}, \qquad \mathrm{tr}(M) = 5 + 3 = 8,$$

we simply walk down the diagonal and add. The trace only makes sense for square matrices — our 3×4 traffic matrix A has no single "diagonal" running corner to corner, so $\mathrm{tr}(A)$ is undefined. However, multiplying A by its transpose forms a square 3×3 intersection co-occupancy matrix $M = AA^\mathsf{T}$:

$$AA^\mathsf{T} = \begin{pmatrix} 30 & 18 & 25 & 12 \\ 22 & 10 & 15 & 8 \\ 40 & 25 & 30 & 20 \end{pmatrix} \begin{pmatrix} 30 & 22 & 40 \\ 18 & 10 & 25 \\ 25 & 15 & 30 \\ 12 & 8 & 20 \end{pmatrix} = \begin{pmatrix} 2023 & 1281 & 2640 \\ 1281 & 873 & 1740 \\ 2640 & 1740 & 3525 \end{pmatrix}.$$

Computing its diagonal sum gives $\mathrm{tr}(AA^\mathsf{T}) = 2023 + 873 + 3525 = 6421$.

Why does this matter? The trace turns out to be quietly connected to the Frobenius norm we just met: for *any* matrix A,

$$\|A\|_F^2 = \mathrm{tr}(AA^\mathsf{T}) = \mathrm{tr}(A^\mathsf{T}A).$$

That is, squaring every entry of A and adding them all up ($30^2 + 18^2 + \cdots + 20^2 = 6421$) is exactly the same as first forming AA^T (or $A^\mathsf{T}A$) and then just reading off its diagonal. We will see why once we compute $A^\mathsf{T}A$ for our traffic matrix in Section 5.2.

Check yourself. What is $\mathrm{tr}(I_5)$, the trace of the 5×5 identity matrix?

Answer. 5. The identity has a 1 in every one of its 5 diagonal positions, so the diagonal sum is $1+1+1+1+1=5$.

2.5 Symmetric and Skew-Symmetric Matrices

Some square matrices have a special kind of mirror symmetry across their diagonal. A matrix M is **symmetric** if flipping it (transposing it) leaves it unchanged: $M = M^\mathsf{T}$, i.e. $M_{ij} = M_{ji}$ for every i, j. Picture a matrix recording how "compatible" every pair of approaches is with each other — compatibility between North and East should be the same number as compatibility between East and North, so this matrix is naturally symmetric:

$$M = \begin{pmatrix} 0 & 3 & 1 & 2 \\ 3 & 0 & 4 & 1 \\ 1 & 4 & 0 & 2 \\ 2 & 1 & 2 & 0 \end{pmatrix}, \qquad M^\mathsf{T} = M.$$

A matrix is **skew-symmetric** instead if transposing it flips every sign: $M = -M^\mathsf{T}$, i.e. $M_{ij} = -M_{ji}$. This naturally describes *directional* relationships — for instance, the net flow of vehicles cutting from approach i to approach j during a cycle, where a flow from North to East should be exactly the negative of the flow from East to North:

$$N = \begin{pmatrix} 0 & 6 & -2 \\ -6 & 0 & 3 \\ 2 & -3 & 0 \end{pmatrix}, \qquad N^\mathsf{T} = -N.$$

Notice a skew-symmetric matrix's diagonal is forced to be all zeros, since $M_{ii} = -M_{ii}$ only holds when $M_{ii} = 0$.

Every square matrix, symmetric or not, can be split into a symmetric part and a skew-symmetric part:

$$M = \underbrace{\tfrac{1}{2}(M + M^\mathsf{T})}_{\text{symmetric}} + \underbrace{\tfrac{1}{2}(M - M^\mathsf{T})}_{\text{skew-symmetric}}.$$

Decomposing Inter-Intersection Traffic Interactions. Using the 3×3 co-occupancy matrix AA^T derived directly from our primary traffic grid A:

$$AA^\mathsf{T} = \begin{pmatrix} 2023 & 1281 & 2640 \\ 1281 & 873 & 1740 \\ 2640 & 1740 & 3525 \end{pmatrix}.$$

Because AA^T is inherently symmetric ($(AA^\mathsf{T})^\mathsf{T} = AA^\mathsf{T}$), its decomposition yields $S = \frac{1}{2}(AA^\mathsf{T} + (AA^\mathsf{T})^\mathsf{T}) = AA^\mathsf{T}$ and $K = \frac{1}{2}(AA^\mathsf{T} - (AA^\mathsf{T})^\mathsf{T}) = \mathbf{0}$. This reflects pure bidirectional co-occupancy between intersections without directional bias.

To see a non-zero skew-symmetric component, suppose sensors observe directional traffic interaction $A_{\text{flow}} \in \mathbb{R}^{3\times 3}$ built around AA^T:

$$A_{\text{flow}} = \begin{pmatrix} 2023 & 1300 & 2600 \\ 1262 & 873 & 1750 \\ 2680 & 1730 & 3525 \end{pmatrix}.$$

Decomposing A_{flow} isolates two clean physical meanings:

$$S = \tfrac{1}{2}(A_{\text{flow}} + A_{\text{flow}}^\mathsf{T}) = \begin{pmatrix} 2023 & 1281 & 2640 \\ 1281 & 873 & 1740 \\ 2640 & 1740 & 3525 \end{pmatrix} = AA^\mathsf{T},$$

$$K = \tfrac{1}{2}(A_{\text{flow}} - A_{\text{flow}}^{\mathsf{T}}) = \begin{pmatrix} 0 & 19 & -40 \\ -19 & 0 & 10 \\ 40 & -10 & 0 \end{pmatrix}.$$

Here S, the symmetric part, recovers our exact two-way co-occupancy volume AA^{T} between each pair of intersections. K, the skew-symmetric part, isolates the *net directional traffic drift* (e.g., $K_{12} = 19$ indicates a net traffic movement from Intersection 1 to Intersection 2).

This decomposition will come up again shortly: whenever we multiply any matrix by its own transpose, as in $A^{\mathsf{T}}A$, the result is *always* symmetric — a fact worth remembering before we meet the Gram matrix in Section 5.2.

Check yourself. Is the sum of two symmetric matrices always symmetric?

Answer. Yes. If $M = M^{\mathsf{T}}$ and $N = N^{\mathsf{T}}$, then $(M+N)^{\mathsf{T}} = M^{\mathsf{T}} + N^{\mathsf{T}} = M + N$, so $M + N$ equals its own transpose.

3 Understanding Matrix-Vector Multiplication

The product $y = Ax$ is one of the most vital operations in AI. Depending on how you look at it, it performs two completely different, useful tasks.

3.1 Perspective 1: Row-wise Scoring and Aggregation

Imagine we want to calculate the total number of idling cars at each intersection. We can multiply matrix A by an all-ones vector $x = (1,1,1,1)^{\mathsf{T}}$:

$$Ax = \begin{pmatrix} 30 & 18 & 25 & 12 \\ 22 & 10 & 15 & 8 \\ 40 & 25 & 30 & 20 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 30(1) + 18(1) + 25(1) + 12(1) \\ 22(1) + 10(1) + 15(1) + 8(1) \\ 40(1) + 25(1) + 30(1) + 20(1) \end{pmatrix} = \begin{pmatrix} 85 \\ 55 \\ 115 \end{pmatrix}.$$

Each entry in the output vector is the dot product of a matrix row with vector x.

3.2 Perspective 2: Column-wise Pattern Blending

Now view Ax from the perspective of A's columns ($a_1, a_2, a_3, a_4 \in \mathbb{R}^3$):

$$Ax = x_1 a_1 + x_2 a_2 + x_3 a_3 + x_4 a_4.$$

Here, Ax is a linear combination of the column patterns weighted by the components of x. If a_1, a_2, a_3 represent fundamental traffic profiles ("Morning Rush", "Evening Rush", "School Drop-off"), then vector $x = (0.7, 0.1, 0.2)^{\mathsf{T}}$ blends these patterns to generate a predicted city-wide traffic state.

Check yourself. What happens if you multiply A by $x = (1,0,0,0)^{\mathsf{T}}$?

Answer. You extract the first column of A ($(30, 22, 40)^{\mathsf{T}}$), isolating the Northbound traffic counts across all three intersections.

3.3 Perspective 3: Feature Matrices and Weight Vectors in Machine Learning

Matrix-vector multiplication is also exactly what happens, at scale, inside a machine learning model. Recall from Lecture 1 that for a *single* intersection, a learned weight vector $w = (0.6, 0.3, 0.5, 0.2)^\mathsf{T}$ and intercept $b = 4$ predicted delay via $\hat{y} = w^\mathsf{T}x + b$.

Now stack every intersection's queue vector as a row of a **feature matrix** $X \in \mathbb{R}^{m\times n}$ — exactly our matrix A, reinterpreted: row i is "sample i's features," and column j is "feature j across all samples." Multiplying by the same weight vector scores every sample in one shot:

$$Xw + b\mathbf{1} = \begin{pmatrix} 30 & 18 & 25 & 12 \\ 22 & 10 & 15 & 8 \\ 40 & 25 & 30 & 20 \end{pmatrix} \begin{pmatrix} 0.6 \\ 0.3 \\ 0.5 \\ 0.2 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \\ 4 \end{pmatrix} = \begin{pmatrix} 38.3 \\ 25.3 \\ 50.5 \end{pmatrix} + \begin{pmatrix} 4 \\ 4 \\ 4 \end{pmatrix} = \begin{pmatrix} 42.3 \\ 29.3 \\ 54.5 \end{pmatrix}.$$

This is precisely Perspective 1 (row-wise scoring) applied to all three intersections simultaneously: each output entry is one row of X dotted with w. This $Xw + b$ pattern — a feature matrix times a weight vector, plus a bias — is the core computation inside linear regression and every single layer of a neural network.

Check yourself. In the feature matrix X above, what does column 3 represent, and what would it mean if that column's weight 0.5 were increased?

Answer. Column 3 is the Southbound queue length at each intersection. Increasing its weight would make the model's predicted delay more sensitive to southbound congestion specifically, raising every intersection's predicted delay in proportion to how much southbound traffic it has.

4 Matrix Rank, Column Space, and System Reachability

In Section 3.2, we saw that matrix-vector multiplication Ax forms a linear combination of columns weighted by the elements of x:

$$Ax = x_1a_1 + x_2a_2 + \cdots + x_na_n.$$

This reveals a fundamental system property:
The output vector $y = Ax$ can only ever be a blend of the column patterns inside A. This leads to a critical question for AI models and physical systems alike: *What is the full range of outputs a matrix A can actually generate, and do all of its columns contribute unique information?*

4.1 The Column Space: Everything Matrix A Can Reach

To examine column space with 4-dimensional approach vectors, consider $A^\mathsf{T} \in \mathbb{R}^{4\times 3}$ whose three columns ($a_1, a_2, a_3 \in \mathbb{R}^4$) represent intersection profile vectors across the four approaches (North, East, South, West):

$$A^\mathsf{T} = \begin{pmatrix} a_1 & a_2 & a_3 \end{pmatrix} = \begin{pmatrix} 30 & 22 & 40 \\ 18 & 10 & 25 \\ 25 & 15 & 30 \\ 12 & 8 & 20 \end{pmatrix}.$$

The set of *all possible outputs* $y = A^\mathsf{T}x$ that can be generated by varying the input weights $x = (x_1, x_2, x_3)^\mathsf{T}$ is called the **Column Space** (or **Range**) of the matrix, written $\mathcal{C}(A^\mathsf{T})$:

$$\mathcal{C}(A^\mathsf{T}) = \{y \in \mathbb{R}^4 \mid y = A^\mathsf{T}x \text{ for some } x \in \mathbb{R}^3\} = \text{span}(a_1, a_2, a_3).$$

Physical Meaning in AI Systems:

- **Reachable States:** If a target network congestion state y_{target} lies *inside* $\mathcal{C}(A^\mathsf{T})$, then there exists a valid control signal x such that $A^\mathsf{T}x = y_{\text{target}}$.
- **Unreachable States:** If y_{target} lies *outside* $\mathcal{C}(A^\mathsf{T})$ (for example, an abnormal traffic queue spike caused by a sudden road closure that does not resemble any column pattern in A^T), no input x can ever produce y_{target}. The system $A^\mathsf{T}x = y_{\text{target}}$ has **no exact solution**.

4.2 Matrix Rank and Column Redundancy

Suppose an automated monitoring system appends a fourth column $a_4 = a_1 + a_2$ to matrix A^T, creating a 4×4 matrix $A^\mathsf{T}_{\text{ext}}$:

$$A^\mathsf{T}_{\text{ext}} = \begin{pmatrix} a_1 & a_2 & a_3 & (a_1 + a_2) \end{pmatrix}.$$

Does adding this column expand the set of reachable outputs $\mathcal{C}(A^\mathsf{T}_{\text{ext}})$? **No.** Because a_4 is a duplicate combination of existing columns, any state built using a_4 could be built using only a_1 and a_2. The columns of $A^\mathsf{T}_{\text{ext}}$ are **linearly dependent**:

$$1 \cdot a_1 + 1 \cdot a_2 + 0 \cdot a_3 + (-1) \cdot a_4 = \mathbf{0}.$$

The true expressiveness or "dimensional capacity" of a matrix is captured by its **Rank**:

$$\text{rank}(A) = \text{number of linearly independent columns in } A.$$

- **Full Column Rank ($\text{rank}(A) = n$):** Every column provides independent, non-redundant information. No column can be written as a combination of the others.
- **Rank Deficient ($\text{rank}(A) < \min(m, n)$):** At least one column is redundant. The matrix collapses input directions and destroys information.

Student Habit

The Fundamental Rank Equality: For any matrix $A \in \mathbb{R}^{m \times n}$, the maximum number of independent rows is *always equal* to the maximum number of independent columns:

$$\text{row rank}(A) = \text{column rank}(A) = \text{rank}(A) \leq \min(m, n).$$

4.3 Matrix Basis and Dimension of Column Space

A subset of A's columns forms a **basis** for $\mathcal{C}(A)$ if those columns:

(a) Are linearly independent (no redundant columns), and

(b) Span the entire column space $\mathcal{C}(A)$.

The dimension of $\mathcal{C}(A)$ is exactly $\text{rank}(A)$. If $A^\mathsf{T} \in \mathbb{R}^{4 \times 3}$ has 3 independent columns, then $\text{rank}(A) = 3$. The matrix maps inputs from $\mathbb{R}^3$ into a 3-dimensional subspace embedding within the 4-dimensional output space $\mathbb{R}^4$.

4.4 Connection to Linear Systems and Matrix Inversion

The concepts of Column Space and Rank directly dictate whether a matrix system $Mx = b$ can be solved or inverted:

Check yourself. Suppose a feature matrix $X \in \mathbb{R}^{1000 \times 50}$ collects 50 features for 1000 traffic intersections. If Feature 50 is accidentally entered in exact duplicate of Feature 1, what is the maximum possible rank of X?

System Setting	Rank Condition	Solvability / Invertibility Behavior
Square System ($M \in \mathbb{R}^{n\times n}$)	Full Rank ($\text{rank}(M) = n$)	Unique solution $x = M^{-1}b$ exists for every b.
Square System ($M \in \mathbb{R}^{n\times n}$)	Rank Deficient ($\text{rank}(M) < n$)	Matrix is **singular**. M^{-1} does not exist.
Tall System ($M \in \mathbb{R}^{m\times n}, m > n$)	Full Column Rank ($\text{rank}(M) = n$)	$Mx = b$ solvable iff $b \in \mathcal{C}(M)$. Left-inverse $M^{\dagger}$ exists.
Wide System ($M \in \mathbb{R}^{m\times n}, m < n$)	Full Row Rank ($\text{rank}(M) = m$)	Infinitely many solutions exist. Right-inverse $M^{\dagger}$ exists.

Table 1: How matrix rank dictates system solvability and the existence of inverses.

Answer. $\text{rank}(X) \leq 49$. Because Column 50 is linearly dependent on Column 1, X cannot have full column rank (50). The maximum number of independent feature columns is 49, making $X^{\mathsf{T}}X$ singular and uninvertible without regularization or pseudo-inverses (Section 8.6).

5 Mastering Matrix Multiplication

Matrix multiplication $Y = AS$ applies a set of transformations or evaluations to an entire collection of data vectors at once.

5.1 The Dimension Matching Rule

To multiply two matrices, their dimensions must align naturally.

$$\boxed{(m \times p)(p \times n) = (m \times n)}$$

- **Inner dimensions must match ($p = p$):** The number of columns in the first matrix must match the number of rows in the second.
- **Outer dimensions dictate the result ($m \times n$):** The output takes its row count from the first matrix and its column count from the second.

$$\underbrace{A}_{m\times p}\ \underbrace{S}_{p\times n} = \underbrace{AS}_{m\times n}$$

Valid: $(3 \times 4)(4 \times 2) = (3 \times 2)$. Inner dimensions match $(4 = 4)$.
Invalid: $(4\times 2)(3\times 4)$. Inner dimensions do not match $(2 \neq 3)$, making multiplication impossible.

Student Habit

Before computing any matrix product, write down their dimensions side-by-side to ensure the inner values match!

Suppose traffic authorities want to collapse four lane counts into just two summary metrics for each intersection: **North–South Load** and **East–West Load**.

We design a summary transformation matrix $S \in \mathbb{R}^{4\times 2}$:

$$S = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Multiplying $A \in \mathbb{R}^{3\times 4}$ by $S \in \mathbb{R}^{4\times 2}$ yields a (3×2) summary matrix AS:

$$AS = \begin{pmatrix} 30 & 18 & 25 & 12 \\ 22 & 10 & 15 & 8 \\ 40 & 25 & 30 & 20 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 30+25 & 18+12 \\ 22+15 & 10+8 \\ 40+30 & 25+20 \end{pmatrix} = \begin{pmatrix} 55 & 30 \\ 37 & 18 \\ 70 & 45 \end{pmatrix}.$$

With a single matrix multiplication operation, we transformed all three intersections simultaneously from 4-lane representations to 2-directional loads!

5.2 Comparing Feature Profiles: The Gram Matrix

When we multiply a matrix by its transpose ($G = A^\mathsf{T}A \in \mathbb{R}^{4\times 4}$), we compute the dot products between every pair of columns in A. The resulting square matrix is called a **Gram matrix**.

If the columns of A represent traffic queue patterns across different approaches, $A^\mathsf{T}A$ measures how strongly aligned or correlated those approach patterns are across the whole city. Because $(A^\mathsf{T}A)^\mathsf{T} = A^\mathsf{T}(A^\mathsf{T})^\mathsf{T} = A^\mathsf{T}A$, the Gram matrix is always symmetric — exactly the property from Section 2.5. Its diagonal entries, $\mathrm{tr}(A^\mathsf{T}A) = \|A\|_F^2$, are the squared column norms, tying together the trace, the Frobenius norm, and the Gram matrix into one identity.

6 Matrices as Linear Transformations

Beyond static tables of numbers, a matrix $T \in \mathbb{R}^{m\times n}$ can be understood dynamically as a machine or function $f(x) = Tx$ that accepts an input vector $x \in \mathbb{R}^n$ and transforms it into an output vector $y \in \mathbb{R}^m$.

() The Matrix Equation

$$\begin{pmatrix} 1 & 2 & 1 \\ 2 & -1 & 3 \\ 1 & 1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 3 \\ 3 \end{pmatrix}.$$

() The Vector Equation

$$\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} x_1 + \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix} x_2 + \begin{pmatrix} 1 \\ 3 \\ -1 \end{pmatrix} x_3 = \begin{pmatrix} 4 \\ 3 \\ 3 \end{pmatrix}.$$

6.1 Intuitive Systems Thinking

Input (x): Proposed green-light duration allocations across traffic light phases.
System (T): The physical response dynamics of the network infrastructure.
Output ($y = Tx$): Expected reduction in vehicle queues.

6.2 Common Transformation Behaviors

- **Identity (I):** Leaves inputs completely unchanged ($Ix = x$).
- **Scaling (αI):** Uniformly expands or contracts inputs (e.g., boosting all green times by 10%).
- **Permutation (P):** Rearranges the components of x (e.g., reindexing traffic lanes after a detour setup).
- **Rotation (Q):** Shifts vector directions geometrically in space without changing their lengths.

7 Inverting Systems: Undoing Transformations

If $y = Tx$ models how signal choices x produce queue changes y, a crucial engineering question arises: If we have a desired target queue reduction y, what signal inputs x should we set?

This requires "undoing" the action of T.

7.1 The Matrix Inverse Concept

For certain square matrices M, there exists an inverse matrix M^{-1} such that:

$$M^{-1}M = I \quad \text{and} \quad Mx = y \implies x = M^{-1}y.$$

Multiplying by M^{-1} plays the same role as dividing by a non-zero number in basic algebra.

7.2 Worked Inverse Scenario

Suppose a 2×2 traffic response system is governed by:

$$M = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}.$$

Its inverse matrix happens to be:

$$M^{-1} = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix}.$$

If our target queue reduction goal is $y = (3, 1)^{\mathsf{T}}$, we calculate the exact required green-time setting x via $M^{-1}y$:

$$x = M^{-1}y = \begin{pmatrix} 1 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 1(3) - 1(1) \\ -1(3) + 2(1) \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}.$$

7.3 When Is Inversion Impossible? Understanding Singular Matrices

Not all matrices can be inverted. A matrix can be undone only if its columns are linearly independent — that is, only if it has full rank.

Watch what goes wrong when they aren't. Consider

$$M = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix}.$$

Its second column is just twice its first, so the columns are dependent, and its span is only a 1-dimensional line, not the full 2-dimensional plane. Now try the input $x = (2, -1)^{\mathsf{T}}$:

$$Mx = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 2 \\ -1 \end{pmatrix} = \begin{pmatrix} 1(2) + 2(-1) \\ 2(2) + 4(-1) \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

A *nonzero* input produced the *zero* output! Worse, $x = (0,0)^\mathsf{T}$ also produces $(0,0)^\mathsf{T}$, and in fact adding any multiple of $(2,-1)^\mathsf{T}$ to any input leaves the output completely unchanged. Two genuinely different signal settings are now indistinguishable from the network's perspective — information about which one was actually used has been permanently destroyed. A matrix with this behavior is called **singular**: because it collapses some input directions to zero, there is no way to build an inverse that could ever recover them. Only matrices with full column rank (independent columns) avoid this collapse and remain invertible.

Check yourself. If M is an invertible matrix and $Mx = 0$, what must x be?

Answer. $x = 0$. Multiplying both sides by M^{-1} yields $x = M^{-1}(0) = 0$. Because no information was destroyed by M, only a zero input could have produced a zero output.

7.4 The Determinant: Measuring Area and Volume Scaling

The **determinant**, written $\det(M)$, is a single scalar that measures how much a linear transformation M scales spatial areas (in 2D) or volumes (in 3D).

- For a 2×2 matrix $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$, $\det(M) = ad - bc$.
- **Geometric Meaning:** If $\det(M) = 2$, M doubles spatial area. If $\det(M) = 0$, M compresses 2D space down to a 1D line or point (collapsing area to zero).
- **Invertibility Criterion:** A square matrix M is invertible **if and only if** $\det(M) \neq 0$.

8 Matrix Inverses: Left, Right, and Pseudo-Inverses

We saw that a square matrix $M \in \mathbb{R}^{n \times n}$ with independent columns has a standard two-sided inverse M^{-1} satisfying $M^{-1}M = MM^{-1} = I$. However, real-world data matrices in AI are rarely square! We frequently encounter **tall matrices** (more observations than features, $m > n$) or **wide matrices** (more parameters than constraints, $m < n$).

To "undo" transformations and solve systems when M is non-square, we generalize the concept of inversion to **one-sided inverses** and **pseudo-inverses**.

8.1 Left-Inverse (Tall Matrices: More Equations than Unknowns)

When $M \in \mathbb{R}^{m \times n}$ is "tall" ($m > n$), our traffic control center has more sensor measurements than traffic signals to adjust.

- **Definition & Behavior:** A left-inverse is a matrix $L \in \mathbb{R}^{n \times m}$ that undoes M from the left:
$$LM = I_n.$$
- **Column Independence Connection:** A left-inverse exists *if and only if* the columns of M are linearly independent. By the independence-dimension inequality, we must have $m \geq n$ so that the output space has enough dimensions to preserve all input directions without squashing any to zero.
- **Solving Systems ($Mx = b$):** For a tall matrix, an exact solution x to $Mx = b$ may not exist if b lies outside the column span of M. However, if an exact solution *does* exist, multiplying on the left by L recovers it directly:
$$L(Mx) = Lb \implies (LM)x = Lb \implies I_n x = Lb \implies x = Lb.$$

Consider a single signal setting mapping to two queue sensors $M = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \in \mathbb{R}^{2\times 1}$. The matrix $L = \begin{pmatrix} 0.2 & 0.4 \end{pmatrix} \in \mathbb{R}^{1\times 2}$ is a valid left-inverse because:

$$LM = \begin{pmatrix} 0.2 & 0.4 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = 0.2(1) + 0.4(2) = 1 = I_1.$$

8.2 Right-Inverse (Wide Matrices: More Unknowns than Equations)

When $M \in \mathbb{R}^{m\times n}$ is "wide" ($m < n$), the control center has fewer sensor constraints than signal variables (an underdetermined system).

- **Definition & Behavior:** A right-inverse is a matrix $R \in \mathbb{R}^{n\times m}$ that undoes M when applied from the right:
$$MR = I_m.$$
- **Row Independence Connection:** A right-inverse exists **if and only if the rows of M are linearly independent** (full row rank). Here $n \geq m$, meaning the columns of M fully span the output space $\mathbb{R}^m$.
- **Solving Systems ($Mx = b$):** In an underdetermined system, infinitely many input vectors x yield output b. A right-inverse constructs one guaranteed solution: setting $x = Rb$ gives
$$Mx = M(Rb) = (MR)b = I_m b = b.$$

Consider $M = \begin{pmatrix} 1 & 2 \end{pmatrix} \in \mathbb{R}^{1\times 2}$. The matrix $R = \begin{pmatrix} 0.2 \\ 0.4 \end{pmatrix} \in \mathbb{R}^{2\times 1}$ is a right-inverse because:

$$MR = \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 0.2 \\ 0.4 \end{pmatrix} = 1(0.2) + 2(0.4) = 1 = I_1.$$

8.3 Standard Two-Sided Inverse (Square & Nonsingular)

When $M \in \mathbb{R}^{n\times n}$ is square ($m = n$) and its columns (and rows) are fully independent:

- **Unique System Solution:** For any target vector b, $Mx = b$ has the unique solution $x = M^{-1}b$.
- **Key Properties of Invertible Matrices:**
  - $(M^{-1})^{-1} = M$
  - $(M_1 M_2)^{-1} = M_2^{-1} M_1^{-1}$ (**Note the order reversal!**)
  - $(M^\mathsf{T})^{-1} = (M^{-1})^\mathsf{T}$
  - $\det(M) \neq 0$ (**Non-zero determinant requirement**)

8.4 Special Structured Invertible Matrices

- **Vandermonde Matrix:** A matrix where each row contains powers of a parameter, $V_{ij} = x_i^{j-1}$:
$$V = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \end{pmatrix}.$$
Used in polynomial curve fitting, a square Vandermonde matrix is invertible as long as all evaluation points x_i are distinct.

8.5 Orthogonal and Orthonormal Matrices

Recall from Section 6.2 that a rotation matrix reorients a vector without stretching or shrinking it. Take a 90° rotation in $\mathbb{R}^2$:

$$Q = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

Its columns, $(0,1)^\mathsf{T}$ and $(-1,0)^\mathsf{T}$, are unit length and perpendicular to each other — we call such a set of vectors **orthonormal**: mutually perpendicular *and* each of length exactly 1. A square matrix built entirely from orthonormal columns is called an **orthogonal matrix**, and it satisfies the neat identity $Q^\mathsf{T}Q = QQ^\mathsf{T} = I$. Check it directly for our rotation matrix:

$$Q^\mathsf{T}Q = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I.$$

Student Habit

Computational Shortcut: For an orthonormal matrix Q, $Q^\mathsf{T}Q = I \implies Q^{-1} = Q^\mathsf{T}$. Inverting Q requires zero numerical solver work—simply transpose it!

More generally, a matrix does not need to be square to have **orthonormal columns**: a tall matrix whose columns are mutually perpendicular unit vectors satisfies $Q^\mathsf{T}Q = I$ (though not necessarily $QQ^\mathsf{T} = I$, since Q isn't square). This is exactly what an orthonormal *basis* for a span looks like when written as the columns of a matrix — every direction accounted for, none overlapping, none redundant.

Check yourself. Is a 90° rotation matrix Q singular?

Answer. No. Rotating a vector never collapses it to zero or merges two different vectors into one output — rotation is always fully reversible (by rotating back −90°), which is exactly Q^T. This matches Section 7.3: an orthogonal matrix's columns are, by definition, independent (in fact orthonormal), so it can never be singular.

8.6 The Pseudo-Inverse: A Universal "Best Undo," $M^\dagger$

Square, full-rank matrices always have a two-sided inverse. But what about a tall matrix like our traffic sensor network, or a wide, underdetermined one? The **Moore–Penrose pseudo-inverse**, written $M^\dagger$ (equivalently M^+), generalizes inversion to *any* matrix at all — square or not, tall or wide, full rank or not. Unlike M^{-1}, which simply does not exist unless M is square and nonsingular, $M^\dagger$ always exists, and it reduces to the ordinary inverse whenever M^{-1} does. When M has full column or full row rank, $M^\dagger$ has the closed-form formulas below.

1. Left Pseudo-Inverse (Tall Matrix M with Linearly Independent Columns)

When $M \in \mathbb{R}^{m\times n}$ $(m > n)$ has independent columns, the Gram matrix $M^\mathsf{T}M \in \mathbb{R}^{n\times n}$ is square and invertible (Section 5.2). The left pseudo-inverse is:

$$M^\dagger = (M^\mathsf{T}M)^{-1}M^\mathsf{T}.$$

- **Verification:** $M^\dagger M = (M^\mathsf{T}M)^{-1}M^\mathsf{T}M = I_n$.
- **Why it matters in AI:** This is the exact closed-form solution for **Ordinary Least Squares (Linear Regression)**. When $Mx = b$ has no exact solution, setting $x^* = M^\dagger b = (M^\mathsf{T}M)^{-1}M^\mathsf{T}b$ finds the input x^* that minimizes the squared prediction error $\|Mx - b\|_2^2$.

2. Right Pseudo-Inverse (Wide Matrix M with Linearly Independent Rows)

When $M \in \mathbb{R}^{m \times n}$ $(m < n)$ has independent rows, $MM^{\mathsf{T}} \in \mathbb{R}^{m \times m}$ is square and invertible. The right pseudo-inverse is:

$$M^{\dagger} = M^{\mathsf{T}}(MM^{\mathsf{T}})^{-1}.$$

- **Verification:** $MM^{\dagger} = MM^{\mathsf{T}}(MM^{\mathsf{T}})^{-1} = I_m$.
- **Why it matters in AI:** Out of the infinitely many solutions to $Mx = b$, $x^* = M^{\dagger}b$ isolates the unique solution vector that minimizes the Euclidean norm $\|x\|_2$.

Check yourself. If $M \in \mathbb{R}^{5 \times 2}$ has linearly independent columns, what is the dimension of its pseudo-inverse $M^{\dagger}$, and what is $M^{\dagger}M$?

Answer. $M^{\dagger} \in \mathbb{R}^{2 \times 5}$. Since M is tall with independent columns, $M^{\dagger}$ acts as a left-inverse, so $M^{\dagger}M = I_2$.

9 Summary Map

Network Observations ($A \in \mathbb{R}^{3 \times 4}$) $\longrightarrow$

- **Combine/Scale:** Addition, Transposition, Norms
- **Reveal Structure:** Trace $\mathrm{tr}(AA^{\top})$ & Symmetric/Skew Split (S, K)
- **Aggregate / Blend:** Row Scoring vs. Column Blending (Ax)
- **ML Batch Operations:** Feature Matrix $\times$ Weights $(Xw + b)$
- **Check Capacity:** Column Space, Rank, Determinant $\det(M)$
- **Transform / Reduce:** Matrix Products (AS)
- **Reverse Process:** Inverses $(M^{-1}, Q^{\top})$ and Pseudo-Inverses $(M^{\dagger})$